套接字地址结构：从数据表示、POSIX 契约到 Linux 内核与 Android 实现

> 技术分析。依据：Linux man-pages 6.19（sockaddr(3type)、ip(7)、ipv6(7)、unix(7)）· POSIX.1-2024 · RFC 791/4632/8200/4007/6874 · Android 13 源码（bionic 头文件与 AAOS13 内核树 net/socket.c），全部断言标注源码路径。

前言

套接字地址结构是网络编程中同时触及语言标准、系统 ABI、协议规范与内核实现的最小交叉点之一。`connect(2)` 的一个指针转换、`bind(2)` 的一个长度参数，背后是 C 类型系统的历史约束、POSIX 的契约演化、内核的拷贝边界与 IPv6 协议的地址模型。

本书沿四个层次展开：数据表示层（地址与端口的数值本质、字节序、结构体布局），核心结构层（sockaddr_in / sockaddr_in6 / sockaddr_un 的逐字段解析与协议依据），系统契约层（sockaddr 的类型擦除、内核承接路径、sockaddr_storage 的对齐保证），以及实现剖析层（bionic 的头文件组织、宏实现与 Java 层入口）。文中所有结构定义、常量与行为断言均标注来源：Linux man-pages 6.19 的对应手册页、POSIX.1-2024、具体 RFC，以及 Android 13 源码的文件路径与行号（bionic 头文件与 AAOS13 内核树 `net/socket.c`）。

阅读第一部分不需要套接字编程经验；第二、三部分面向具备 C 基础的读者；第四部分的工程规则与综合示例可直接作为代码评审参照。

第 1 章 通信端点的定位模型

1.1 两段式定位

任何一次跨进程通信都要回答两个问题：通信终点在哪台主机上，以及在该主机的哪个服务上。IP 协议负责第一段（全局可路由的主机标识），传输层端口负责第二段（主机内的服务编号）。一个完整的 IPv4 套接字地址因此是二元组（IP 地址, 端口），记法如 127.0.0.1:8080。

Unix 域是同一模型在本机的退化形态：两段合并为一段——文件系统路径在单机命名空间内全局唯一，既定位"主机"（本机）又定位"端点"。IPv6 保留两段式，但在链路本地地址上引入第三段（作用域），第 4 章展开。

1.2 套接字与地址的关系

`socket(2)` 的返回只创建一个无地址的本地端点（内核对象与描述符的配对）；`bind(2)` 将地址关联到端点，`connect(2)` 以对端地址发起连接，`accept(2)` 返回已连接对端的地址。地址因此总是以"参数"形态跨越用户态与内核边界，本书讨论的结构就是这枚参数的载体。

1.3 地址抽象：族标签 + 定位字节串

POSIX 将地址抽象为"前 2 字节族标签（`sa_family_t`）+ 其后按族解释的字节串"。三种常用族的定位信息对比：

族	常量	定位信息	定位依据
Unix 域	AF_UNIX = 1	sun_path（路径或抽象名）	单机文件系统命名空间
IPv4	AF_INET = 2	sin_addr(4B) + sin_port(2B)	全局路由地址 + 传输层端口
IPv6	AF_INET6 = 10	sin6_addr(16B) + sin6_port(2B) + 作用域	全局/链路本地地址 + 端口 + scope

同一字节序列在不同族下的解释完全不同，族值是一切解释的前提——这一约定贯穿全书。

第 2 章 数据表示与 C 结构体布局

2.1 IPv4 地址：32 位数值

点分十进制是 32 位数值的显示格式：127.0.0.1 即字节序列 7F 00 00 00（数值 0x7F000001）。程序内转换由 `inet_pton(AF_INET, ...)` 完成（bionic 实现取自 OpenBSD 上游：`bionic/libc/upstream-openbsd/lib/libc/net/inet_pton.c`），输出为网络字节序的 4 字节。特殊段：127/8 为回环（ip(7)："always refers to the local host via the loopback device"），224.0.0.0~239.255.255.255 为多播。

2.2 端口：16 位编号

端口取值 0~65535。0~1023 为特权端口，ip(7)："The port numbers below 1024 are called privileged ports"，绑定需要 CAP_NET_BIND_SERVICE 能力（限定在其网络命名空间所属的 user namespace 内），以防止非特权进程冒充系统服务。常见保留：22（SSH）、80（HTTP）、443（HTTPS）。

2.3 字节序与网络字节序

多字节整数在内存中的字节排列存在两派：小端（x86、多数 ARM）低有效字节在低地址；大端反之。网络协议统一采用大端（网络字节序），因此**自行书写的**端口等数值必须经 `htons(3)/htonl(3)` 转换后存入结构，ip(7)："the address and the port are always stored in network byte order"。

bionic 将转换实现为编译期宏（`bionic/libc/include/sys/endian.h:59`）：

```
#define htons(x) __swap16(x)
```

小端平台上展开为单条字节交换指令，无函数调用开销。内核 uapi 头文件进一步把字节序写进类型名：`__be16` / `__be32`（big-endian 16/32）直接标注 sin_port 等成员。三个易错点：

1. `inet_pton` / `inet_aton` 的输出已是网络序，不得再套 htonl；
2. INADDR_ANY 与 INADDR_BROADCAST 字节序无关（全 0 / 全 1 的对称性使转换无效果），是例外而非常规；
3. 历史 API `inet_addr(3)` 对 255.255.255.255 返回与失败相同的 INADDR_NONE(-1)，存在歧义，新代码应使用 inet_pton。

2.4 结构体布局与类型宽度

C 结构体的三条布局事实是后续所有讨论的基础：

1. 成员按声明顺序、地址递增排列，中间可能插入对齐填充字节；
2. **任何结构体的首成员偏移量恒为 0**（C 标准保证），这使"指向结构体的指针"与"指向首成员的指针"数值相同；
3. 指针 = 地址数值 + 编译期类型标签；指针转换（cast）只改写标签，不触碰内存。

地址结构使用的基础 typedef 及其宽度（bionic 核验）：

类型	定义	宽度
sa_family_t	unsigned short（`bits/sa_family_t.h:39`）	2 字节
in_port_t	uint16_t（`netinet/in.h:55`）	2 字节
in_addr_t	uint32_t（`bits/in_addr.h:40`）	4 字节
socklen_t	uint32_t（`sys/types.h:117`）	4 字节

sockaddr(3type) 对 socklen_t 的定义："an integer type of at least 32 bits"。

第 3 章 sockaddr_in：IPv4 域

3.1 定义原文

Android 13 用户态可见的内核 uapi 定义（`bionic/libc/kernel/uapi/linux/in.h:193`）：

```
#define __SOCK_SIZE__ 16
struct sockaddr_in {
  __kernel_sa_family_t sin_family;
  __be16 sin_port;
  struct in_addr sin_addr;
  unsigned char __pad[__SOCK_SIZE__ - sizeof(short int)
                      - sizeof(unsigned short int) - sizeof(struct in_addr)];
};
#define sin_zero __pad
```

总长 2+2+4+8 = 16 字节。`__pad` 的算式直接表达设计意图：将结构补齐到 `__SOCK_SIZE__` 16 字节。

3.2 字段语义

1. **sin_family**：恒为 AF_INET，ip(7) 注明 "This is required"。
2. **sin_port**：16 位端口，网络字节序。raw socket 上该字段承载 IP 协议号而非端口（ip(7)："On raw sockets .sin_port is set to the IP protocol"）。
3. **sin_addr**：32 位 IPv4 地址，网络字节序；解析经 inet_pton/inet_aton。
4. **sin_zero（__pad）**：8 字节填充，早期实现将 sockaddr_in 与 sockaddr 直接互相赋值的兼容产物；现代用法仅要求置零。

3.3 特殊地址与绑定规则

uapi 常量（`linux/in.h:225`）：INADDR_ANY（0x00000000）、INADDR_LOOPBACK（0x7f000001）、INADDR_BROADCAST / INADDR_NONE（0xffffffff）。绑定语义（ip(7)）：

1. INADDR_ANY："means any address for socket binding"，绑定后关联所有本地接口；
2. 唯一性："only one IP socket may be bound to any given local (address, port) pair"；SO_REUSEADDR 可放宽但 "it makes TCP less reliable"；
3. 隐式绑定：未 bind 直接 listen/connect 时，内核自动绑定随机空闲端口，本地地址取 INADDR_ANY。

3.4 in_addr 的结构外壳：分类编址遗迹

`struct in_addr` 包裹单一成员 `in_addr_t s_addr` 的写法源于 RFC 791 的分类编址（classful）：A/B/C 类的网络号-主机号边界落在地址内部，早期实现预留结构外壳以支持位段访问；RFC 4632 的 CIDR 以任意长度前缀取代分类后外壳失去用途，但 ABI 冻结使其永久保留。工程规则：将 in_addr 视为 32 位整数使用。

3.5 装填演示与逐字节布局

```
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));                /* 置零 sin_zero */
addr.sin_family      = AF_INET;                /* 族标签 */
addr.sin_port        = htons(8080);            /* 0x1F90 → 大端 */
addr.sin_addr.s_addr = inet_addr("127.0.0.1");  /* 已是网络序 */
connect(fd, (const struct sockaddr *)&addr, sizeof(addr));
```

小端机上 addr 的 16 字节内存布局：

```
偏移:  0    1    2    3    4    5    6    7   |  8 ····················· 15
字节:  02   00   1F   90   7F   00   00   00   |  00 ×8
       └sin_family┘ └sin_port┘ └─────sin_addr──────┘   |   └──sin_zero──┘
```

02 00 为 AF_INET=2 的小端拆分；1F 90 为 htons(8080) 结果；7F 00 00 00 为 127.0.0.1；填充为零。最后两行的转换与长度即第 6 章的调用契约。

第 4 章 sockaddr_in6：IPv6 域

4.1 定义原文

uapi 定义（`bionic/libc/kernel/uapi/linux/in6.h:41`）：

```
struct in6_addr { __u8 s6_addr[16]; };
struct sockaddr_in6 {
  __kernel_sa_family_t sin6_family;   /* AF_INET6 = 10 */
  __be16 sin6_port;
  __be32 sin6_flowinfo;
  struct in6_addr sin6_addr;
  __u32 sin6_scope_id;
};
```

总长 28 字节。ipv6(7)："The sockaddr_in6 structure is bigger than the generic sockaddr"，接收端必须使用 sockaddr_storage（第 8 章）。

4.2 sin6_flowinfo 与长度契约

承载 IPv6 头部的流标签与流量类别（RFC 8200，流标签 20 位）。ipv6(7) 给出一条精确到调用契约的行为描述："It is transparently passed/read by the kernel when the passed address length contains it"——addrlen 是否覆盖到该字段决定内核是否处理它。这从实现层面印证第 6 章的结论：长度是解释契约的组成部分。应用层一般置 0。

4.3 sin6_scope_id：作用域消歧

IPv6 链路本地地址（fe80::/10）逐链路自动配置，不具全球唯一性：同一主机的两张网卡各自持有 fe80:: 前缀地址，仅凭 128 位地址无法确定出接口。RFC 4007 以作用域分区（scope zone）建模该问题；sin6_scope_id 承载接口索引完成消歧。ipv6(7)："Linux supports it only for link-local addresses, in that case sin6_scope_id contains the interface index"。程序化获

取索引用 `if_nametoindex(3)`；文本形式为 RFC 6874 的 %zone 语法（fe80::1%3）。错误契约：绑定链路本地地址而 scope_id 非有效接口索引时，bind(2) 返回 ENODEV。全球单播场景置 0。

4.4 特殊地址与双栈

`::1` 为回环；`::ffff:a.b.c.d` 为 IPv4-mapped 地址，使 v4 连接可经 v6 API 处理；v4/v6 默认共享本地端口空间，IPV6_V6ONLY 选项控制该行为（详见 ip(7)/ipv6(7)）。

4.5 ABI 断裂史

ipv6(7) 记载 Linux 2.4 曾 "changing the alignment of in6_addr and adding an additional sin6_scope_id field"，在 64 位平台破坏二进制兼容。这证明地址结构并非不可变 ABI；同时解释了当前布局中 flowinfo 与 scope_id 兼任 4 字节对齐填充的成因。

第 5 章 sockaddr_un：Unix 域

5.1 定义原文

uapi 定义（`bionic/libc/kernel/uapi/linux/un.h:22`）：

```
#define UNIX_PATH_MAX 108
struct sockaddr_un {
  __kernel_sa_family_t sun_family; /* AF_UNIX = 1 */
  char sun_path[UNIX_PATH_MAX];
};
```

5.2 三类地址

unix(7) 将地址划分为三类：

1. **pathname**：bind(2) 绑定 NUL 结尾路径，内核创建 socket 类型 inode（`ls -l` 权限位首位 s）。返回地址长度为 `offsetof(struct sockaddr_un, sun_path) + strlen(sun_path) + 1`。创建要求目录的 write+search（execute）权限；socket 文件由调用方 unlink(2) 清理；流套接字 connect 需要该文件写权限（POSIX 未规定，不可作为安全控制依赖）。
2. **unnamed**：未 bind 的套接字与 socketpair(2) 产物；地址长度仅 `sizeof(sa_family_t)`，读取方不应检查 sun_path。
3. **abstract**：`sun_path[0] == '\0'` 标识，非可移植 Linux 扩展（2.2 起）。名字由 addrlen 覆盖的后续字节构成——"the additional bytes in sun_path that are covered by the specified length"——内嵌 NUL 无特殊含义，addrlen 因此参与名字定义；无文件系统对象，权限与 umask 不适用，全部引用关闭后自动消失。Android 系统内部 IPC 广泛使用该命名空间以规避文件残留与文件系统权限耦合。

autobind：bind 传 `sizeof(sa_family_t)`（或对设置 SO_PASSCRED 的未绑定套接字）时，内核自动分配"空字节 + 5 个十六进制字符"的抽象名，上限 2^{20} 个。

5.3 长度边界与可移植性

1. unix(7)："The length of the pathname, including the terminating null byte, should not exceed the size of sun_path"；路径加 NUL 恰为 108 字节时，内核补写终止符会越界，导致 accept/getsockname 返回的 sun_path 无 NUL 终止。安全读取：`strnlen(addr.sun_path, addrlen - offsetof(struct sockaddr_un, sun_path))`，或分配 sizeof(sockaddr_un)+1 的清零缓冲。
2. 108 是 Linux 的 UNIX_PATH_MAX；unix(7) 提示其他实现可能仅 92 字节，硬编码 108 容量不可移植。超长路径 bind 返回 EINVAL；地址被占返回 EADDRINUSE。

第 6 章 sockaddr：类型擦除与调用契约

6.1 问题与方案

C 语言没有继承与接口多态，而 connect(2)/bind(2)/accept(2) 必须以统一原型服务所有地址族。socket API 的方案是**带标签结构的类型擦除**（bionic `sys/socket.h:68`）：

```
struct sockaddr {
    sa_family_t sa_family;
    char        sa_data[14];
};
```

配套两条全局约定：其一，凡地址结构，首成员必须是 2 字节族标签（三种核心结构均遵守）；其二，函数原型统一接受 `struct sockaddr *`，另以 `socklen_t addrlen` 传递真实对象长度：

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

6.2 指针转换的合法性

`connect(fd, (const struct sockaddr *)&addr, sizeof(addr))` 中，cast 只改写指针的编译期类型标签，地址数值与内存内容不变。合法性依据有三层：

1. **C 标准**：结构体首成员偏移恒为 0，两种指针视图在前 2 字节（族字段）上重合；
2. **strict aliasing 历史**：sockaddr(3type) HISTORY 指出这些结构诞生于 ISO C 严格别名规则之前，严格适用别名规则将使它们 "extremely difficult to use without invoking undefined behavior"；
3. **POSIX.1-2024 修复**：标准要求实现确保这些结构可按设计意图安全使用，将长期惯例升格为规范义务。

内核按"族值 + 长度"读取真实对象的分派流程（概念示意）：

```
uint16_t family = *(uint16_t *)addr;          /* 读偏移 0-1 */
switch (family) {
case AF_INET:   /* 校验 addrlen ≥ 16；端口=偏移 2，地址=偏移 4 */
case AF_INET6:  /* 校验 addrlen ≥ 28；地址=偏移 8，作用域=偏移 24 */
case AF_UNIX:   /* 路径=偏移 2 起，addrlen-2 字节 */
}
```

6.3 两个误解与一个陷阱

1. **sa_data[14] 不是容量上限**：sockaddr_in6 为 28 字节，超出 sockaddr 的 16 字节，但内核从不把数据装入 sockaddr，而是直接按族值与长度读取真实对象；14 只是使 sockaddr 与早期 sockaddr_in 等长（2+14=16）的历史尺寸。真实容量上限是 128 字节（第 7 章）。
2. **不需要构造通用对象再抄写 sa_data**：唯一正确形态是"填充具体族结构 + 指针转换 + sizeof 长度"；抄写不仅多余，且物理上装不下。
3. **sizeof 陷阱**：addrlen 必须传 `sizeof(结构变量)`，不能传 `sizeof(&变量)`（64 位平台为 8）；传短导致内核判定数据不足并返回 EINVAL，且编译期不可见。

6.4 BSD 的 sa_len 路线

BSD 系实现把长度嵌入结构开头（sa_len 位于偏移 0，族字段退居偏移 1），对象自描述；Linux/POSIX 把长度作为函数参数。两条 ABI 路线并存，因此跨平台代码不得手工构造结构字节，也不得假定族字段偏移。

第 7 章 Linux 内核的承接路径

7.1 move_addr_to_kernel

AAOS13 内核树 `net/socket.c:242`：

```
int move_addr_to_kernel(void __user *uaddr, int ulen, struct sockaddr_storage *kaddr)
{
        if (ulen < 0 || ulen > sizeof(struct sockaddr_storage))
                return -EINVAL;
        if (ulen == 0)
                return 0;
        if (copy_from_user(kaddr, uaddr, ulen))
                return -EFAULT;
        return audit_sockaddr(ulen, kaddr);
}
```

三点结论：

1. 用户态地址的真实容量上限是 `sizeof(struct sockaddr_storage)` = 128 字节，与 sockaddr 的 16 字节无关——sa_data[14] 不构成约束的内核侧证据；
2. 拷贝严格按 ulen 字节进行，长度是解释契约的组成部分；
3. `audit_sockaddr` 在拷贝完成后对地址执行审计（SELinux 体系），地址进入内核的第一站即被记录。

7.2 move_addr_to_user 与值-结果语义

反方向函数的内核注释（`net/socket.c`）："The value pointed to by ulen on entry is the buffer length available. This is overwritten with the buffer space used."——入参为调用方缓冲容量，出参改写为实际使用长度。accept(2)、getsockname(2)、recvfrom(2) 的 `addrlen` 参数同属**值-结果（value-result）**语义，接收方必须以返回长度为准。

第 8 章 sockaddr_storage 与地址接收

8.1 对齐机制

bionic 定义（`sys/socket.h:73`）：

```
struct sockaddr_storage {
  union {
    struct {
      sa_family_t ss_family;
      char __data[128 - sizeof(sa_family_t)];
    };
    void* __align;
  };
};
```

两个保证：容量 128 字节覆盖任何族；`void *` 成员使联合体取得指针级（平台最严格之一）对齐。sockaddr(3type)："can be cast as a pointer to other sockaddr_* structures and used to access its fields"。普通 `char buf[128]` 仅保证 1 字节对齐，转型后访问 4 字节成员违反对齐要求，在严格对齐架构上是未定义行为——这正是必须使用 sockaddr_storage 的原因。

8.2 接收范式

```
struct sockaddr_storage ss;
socklen_t len = sizeof(ss);                    /* 入参：容量 */
int cfd = accept(lfd, (struct sockaddr *)&ss, &len);
/* len（出参）= 实际地址长度；按族分派解释 */
if (ss.ss_family == AF_INET6) {
    struct sockaddr_in6 *p6 = (struct sockaddr_in6 *)&ss;
    /* 读 p6->sin6_port / p6->sin6_addr / p6->sin6_scope_id */
}
```

三步：容量给足、以返回长度为准、按 ss_family 分派。多开读端各自建立独立 reader 语义与此无关，此处仅指单进程多次 accept 的通用写法。

第 9 章 Android/bionic 实现剖析

9.1 头文件的三层来源

1. **bionic 自有头**：sockaddr、sockaddr_storage、AF/PF 常量（`libc/include/sys/socket.h:68,73,146`）与字节序宏（`libc/include/sys/endian.h:59`）；
2. **内核 uapi 同步生成**：sockaddr_in/in6/un 的定义来自 `libc/kernel/uapi/linux/{in,in6,un}.h`，与内核 ABI 严格一致——`__be16/__be32` 字节序标注与 `__pad` 算式均直接出自内核头；
3. **OpenBSD 上游**：inet_pton 等解析函数（`libc/upstream-openbsd/lib/libc/net/inet_pton.c`）。

9.2 实现要点

1. **字节序原语是宏**：htons/htonl 展开为 `__swap16/__swap32`，编译期内联为字节交换指令；内核侧对应 `cpu_to_be16` 族转换。
2. **内核承接无平台差异**：AAOS13 内核的 move_addr_to_kernel（128 上限、audit_sockaddr）与上游 Linux 一致。
3. **Java 层入口**：应用以 `java.net.InetSocketAddress` 描述地址，libcore 的 `libcore/io`（Os.bind/connect/accept）在 native 层完成与 sockaddr 的互转；Java 层不直接操作结构内存。
4. **平台场景**：Android 系统进程间本地 IPC 大量使用 Unix 域抽象命名空间（第 5.2 节），利用其无文件残留、生命周期随套接字关闭终止的特性。

第 10 章 其他地址族

同一"族值 + 长度 + 布局"契约承载的其他常见族：

结构	族	定位字段	源码
sockaddr_ll	AF_PACKET	sll_ifindex、sll_protocol、sll_halen+sll_addr[8]	uapi/linux/if_packet.h
sockaddr_nl	AF_NETLINK	nl_pid（默认进程 PID）、nl_groups（多播组位图）	uapi/linux/netlink.h

Android 平台上 AF_NETLINK（rtnetlink 网络状态监控）与 AF_PACKET（抓包工具）经完全相同的 cast + addrlen 契约使用，印证抽象的普适性。AF 常量清单起于 `bionic/libc/include/sys/socket.h:146`（AF_UNSPEC=0、AF_UNIX=AF_LOCAL=1、AF_INET=2 …）。

第 11 章 工程规则与错误模式

1. **传参方向**：填充具体族结构，指针转换后连同 `sizeof(结构变量)` 传入；禁止手工构造 sockaddr 或抄写 sa_data（依据：第 6.3、7.1 节）。
2. **接收方向**：sockaddr_storage 缓冲，容量经 socklen_t 传入，以返回长度与 ss_family 为准（第 8 章）。
3. **字节序**：自行书写的多字节整数一律 htonl/htons；inet_pton/inet_aton 结果已为网络序；INADDR_ANY/BROADCAST 为序无关特例（第 2.3 节）。
4. **置零**：结构先 memset 再填充，保证 sin_zero、sin6_flowinfo、sin6_scope_id 确定（第 3.2、4.2 节）。
5. **链路本地 IPv6**：必须填 if_nametoindex 的接口索引，否则 bind 返回 ENODEV（第 4.3 节）。
6. **Unix 域长度**：路径 + NUL ≤ sizeof(sun_path)；读取对端路径用 strnlen 截断，不依赖 NUL 终止（第 5.3 节）。
7. **addrlen 纪律**：永远传 sizeof(结构变量)；内核真实上限 128 字节（第 7.1 节）。
8. **可移植性**：不假定族字段偏移（BSD sa_len）与 sun_path 容量（92/104/108 各异），以目标系统头文件为准（第 5.3、6.4 节）。

第 12 章 综合示例

12.1 IPv4 TCP 客户端的地址装填

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>

int tcp_connect_ipv4(const char *ip, uint16_t port) {
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));           /* 置零填充字段 */
    addr.sin_family = AF_INET;                /* 族标签 */
    addr.sin_port   = htons(port);            /* 端口转网络序 */
    if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
        return -1;                             /* 写法非法 */
    }
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    if (connect(fd, (const struct sockaddr *)&addr, sizeof(addr)) < 0) {
        close(fd);
        return -1;                             /* 转换 + sizeof 长度 */
    }
    return fd;
}
```

要点：memset 全量置零、inet_pton 校验返回值、connect 的第二参为转换指针、第三参为 sizeof(addr)。

12.2 双栈服务端的 accept 分派

```
struct sockaddr_storage ss;
socklen_t len = sizeof(ss);
int cfd = accept(lfd, (struct sockaddr *)&ss, &len);   /* len 值-结果 */
if (cfd >= 0) {
    char host[INET6_ADDRSTRLEN];
    if (ss.ss_family == AF_INET) {
        struct sockaddr_in *p4 = (struct sockaddr_in *)&ss;
        inet_ntop(AF_INET, &p4->sin_addr, host, sizeof(host));
        printf("v4 %s:%u\n", host, ntohs(p4->sin_port));
    } else if (ss.ss_family == AF_INET6) {
        struct sockaddr_in6 *p6 = (struct sockaddr_in6 *)&ss;
        inet_ntop(AF_INET6, &p6->sin6_addr, host, sizeof(host));
        printf("v6 %s:%u scope=%u\n", host, ntohs(p6->sin6_port),
               p6->sin6_scope_id);
    }
}
```

要点：sockaddr_storage 承接、len 以返回值为准、按 ss_family 分派后再访问族内字段、打印端口用 ntohs。

12.3 Unix 域抽象命名空间服务端

```
#include <sys/un.h>

int unix_abstract_listen(const char *name) {   /* name 不含前导 NUL */
    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    addr.sun_path[0] = '\0';                   /* abstract 标识 */
    strncpy(addr.sun_path + 1, name, sizeof(addr.sun_path) - 2);
    socklen_t len = offsetof(struct sockaddr_un, sun_path) + 1 + strlen(name);
    int fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0) return -1;
    if (bind(fd, (struct sockaddr *)&addr, len) < 0) {  /* 长度参与命名 */
        close(fd);
        return -1;
    }
    listen(fd, 16);
    return fd;
}
```

要点：sun_path[0] 置 '\0' 进入抽象命名空间；addrlen 按 offsetof + 1 + 名长计算（名字由覆盖字节定义）；strncpy 上限留出首字节与终止符；无文件系统对象，无需 unlink。

附录 A 结构与常量速查

结构	族值	总长	关键字段	定义源码
sockaddr	—	16	sa_family, sa_data[14]	bionic sys/socket.h:68
sockaddr_in	2	16	sin_family/sin_port/sin_addr/sin_zero	uapi linux/in.h:193
sockaddr_in6	10	28	sin6_family/sin6_port/sin6_flowinfo/sin6_addr/sin6_scope_id	uapi linux/in6.h:41
sockaddr_un	1	≤110	sun_family/sun_path[108]	uapi linux/un.h:22
sockaddr_storage	—	128	ss_family + __data（指针级对齐）	bionic sys/socket.h:73
sockaddr_ll	AF_PACKET	20	sll_ifindex/sll_halen/sll_addr[8]	uapi linux/if_packet.h
sockaddr_nl	AF_NETLINK	12	nl_pid/nl_groups	uapi linux/netlink.h

常量：AF_UNSPEC=0、AF_UNIX=AF_LOCAL=1、AF_INET=2；UNIX_PATH_MAX=108（Linux）；INADDR_ANY=0、INADDR_LOOPBACK=0x7f000001、INADDR_BROADCAST=INADDR_NONE=0xffffffff；__SOCK_SIZE__=16。

附录 B 错误码对照

错误码	触发场景	出处
EINVAL	addrlen < 0 或 > 128；Unix 域路径超长	内核 net/socket.c；unix(7)
EFAULT	copy_from_user 失败（指针不可访问）	内核 net/socket.c
ENODEV	bind 链路本地 IPv6 而 scope_id 非有效接口索引	ipv6(7)
EADDRINUSE	(地址, 端口) 已被占用；Unix 路径已有 socket 文件	ip(7)；unix(7)
EACCES	非特权进程绑定 0~1023 端口（需 CAP_NET_BIND_SERVICE）	ip(7)

附录 C 依据清单

1. **手册页**（Linux man-pages 6.19）：sockaddr(3type)（类型语义、strict aliasing 与 POSIX.1-2024 修复）；ip(7)（网络序、INADDR 语义、特权端口、绑定规则、raw socket）；ipv6(7)（结构尺寸、flowinfo 长度契约、scope_id 与 ENODEV、Linux 2.4 ABI）；unix(7)（三类地址、abstract 与 autobind、长度边界与错误）。
2. **协议标准**：RFC 791（IPv4 与分类编址）、RFC 4632（CIDR）、RFC 8200（IPv6 与流标签）、RFC 4007（IPv6 作用域地址）、RFC 6874（%zone 文本语法）。
3. **Android 13 源码**：`bionic/libc/include/sys/socket.h:68,73,146`；`bionic/libc/include/sys/endian.h:59`；`bionic/libc/kernel/uapi/linux/in.h:193-230`；`bionic/libc/kernel/uapi/linux/in6.h`；`bionic/libc/kernel/uapi/linux/un.h:22-26`；`bionic/libc/upstream-openbsd/lib/libc/net/inet_pton.c`；AAOS13 内核树 `net/socket.c:232-262`（move_addr_to_kernel / move_addr_to_user）。